# LifeAdmin: A Personal Administrative Management Platform

Project Proposal for UCS503P
Group 3C23
Submitted to: Dr. Raghav B. Venkataramaiyer
**Thapar Institute of Engineering and Technology**

Kushant Garg, COE* Sanvi, COE† Loveleen Singh, COE‡

September 6, 2026

## 1 Higher-order goal

*. . . secondary framing*

This project aims to reduce the cognitive burden of everyday personal administration: deadlines, renewals, and small recurring tasks that are easy to forget but costly to miss. While document-locker services such as DigiLocker solve the problem of *storing* documents, they do not solve the problem of *acting* on the obligations tied to those documents. The immediate engineering objective is to translate *proactive task and deadline visibility* into a measurable, deployable software solution that complements, rather than replaces, existing document-storage infrastructure.

## 2 Time-to-value

*. . . primary heuristic*

- We will deliver meaningful increments quickly by prototyping the core add-task/get-reminded workflow first and validating it with a small group of peer users within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration – a working deadline tracker with reliable reminders – then improved based on user feedback.

## 3 Problem Statement

Individuals routinely lose track of administrative deadlines and renewals – ID and license renewals, subscription cycles, form submissions, fee payments, and similar recurring obligations. Common pain points include:

- **Fragmentation:** obligations are scattered across emails, paper notices, apps, and memory, with no single place to see what is due.

*Roll No: 1024030336
†Roll No: 1024031191
‡Roll No: 1024030323

- **Document-action gap:** platforms like DigiLocker give access to a document but do not track or remind about the action required around it (e.g., an expiry date).
- **Reactive, not proactive, awareness:** people typically discover a missed deadline only after a penalty, late fee, or service disruption occurs.
- **Poor task-metadata linkage:** even when reminders exist (calendar entries, notes apps), they are not linked to the relevant service, deadline type, or renewal cadence, so they are easy to dismiss or mis-set.

**Why this matters now (contextual relevance):** As more services move to digital renewal cycles with strict, non-negotiable deadlines (and increasing penalties for lapses), a lightweight system that tracks obligations *without* taking on the liability of storing sensitive documents fills a real, currently unaddressed gap for individual users.

# 4 Proposed Solution

*. . . Software Proposition*

## 4.1 Overview

Build a **Java desktop application** (JavaFX front end, JDBC/SQLite persistence) for personal administrative management, with the following components:

- **Administrative dashboard:** a single view of all upcoming and overdue deadlines, renewals, and tasks.
- **Deadline and renewal tracking:** users log obligations (name, category, due date, recurrence), entered manually or auto-filled from a scanned document (see below).
- **Reminders:** configurable, recurring reminders as a deadline approaches, surfaced in-app.
- **Task management:** lightweight sub-tasks attached to a deadline (e.g., "collect document", "pay fee", "submit form").
- **Optional document upload with automatic date detection:** a user may optionally upload a scan/photo of a supporting document (e.g., a driving licence). For supported, pre-templated document types, the system crops a known region of the image where the relevant date is typically printed and runs OCR on just that region to suggest a due date, which the user must confirm before a chore is created. Manual entry always remains available and is the default path for unsupported document types.
- **Document/service metadata:** issuing authority, service name, renewal cadence, and a link to the official portal.
- **Links to official services:** deep-link out to the relevant government/service portal (or DigiLocker itself) for the actual transaction.

## 4.2 Core workflow

1. User adds an obligation either manually (e.g., "Renew driving licence") with a due date and optional recurrence, or by uploading a photo/scan of a supported document type.
2. If a document was uploaded, the system crops the pre-defined date region for that document type, runs OCR, and shows the detected date back to the user for confirmation before creating the obligation; unsupported document types or failed reads fall back to manual entry.

3. System schedules reminders at configurable lead times (e.g., 30/7/1 days before).
4. User receives reminders and marks sub-tasks/steps complete as they progress.
5. On completion, the system either archives the obligation or, if recurring, automatically schedules the next cycle.

### 4.3 Operational constraints for an educational setting

- Keep the solution usable on a standard desktop environment via a straightforward JavaFX interface.
- Only OCR pre-defined regions of a small, explicitly supported set of document layouts (starting with a driving licence template); do not attempt general-purpose document understanding.
- Minimize retention of uploaded documents: use an upload only to extract the needed date, and let the user choose whether the original file is kept or discarded after confirmation.
- Avoid dependence on advanced research algorithms; focus on scheduling correctness, reminder reliability, and clean workflow design.
- Design for deployability as a self-contained, runnable Java application, within our team's growing comfort with Java, JDBC, and JavaFX.

## 5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on scheduling correctness, notification reliability, and workflow design rather than novel algorithm research.

### 5.1 Reminder scheduling *... non-research*

Reminder generation will be rule-based, not predictive:

- Each obligation has a due date and a small set of configurable lead-time offsets (e.g., 30/7/1 days).
- A background scheduler (cron-style job) periodically checks for obligations crossing a lead-time threshold and generates reminders.
- Recurring obligations regenerate their next due date automatically once marked complete, based on a stored recurrence rule (weekly/monthly/yearly).

### 5.2 Application architecture *... desktop-based solution*

A layered desktop application:

- **Frontend (JavaFX):** the dashboard, obligation entry form, document-upload/confirmation dialog, and task checklist views.
- **Domain/business logic (plain Java):** obligation/task management, reminder scheduling logic, recurrence handling, and the OCR-to-obligation confirmation flow.
- **Persistence (JDBC + SQLite):** store users, obligations, sub-tasks, reminder logs, and service metadata locally.

### 5.3 Document scanning (OCR) assistance *... template-based, non-research*

Automatic date detection is scoped narrowly and deliberately kept simple:

- A small library of document templates is maintained by the team, each specifying where the relevant date is typically printed on that document type (starting with a driving licence).
- On upload, the matching template's region is cropped from the image and passed to an OCR engine (Tess4J, a Java wrapper for Tesseract) rather than OCR-ing the whole document.
- The detected date is always shown to the user for confirmation before an obligation is created; nothing is auto-created without user sign-off.
- Documents without a matching template, or where OCR fails/returns low confidence, fall back directly to the manual entry form – OCR is a convenience layer on top of manual entry, never a replacement for it.

### 5.4 CI and fast delivery enabler

Continuous integration will be used to:

- Automatically build and run tests (including scheduling-logic and OCR-region unit tests) on every change.
- Produce a repeatable, runnable build (a packaged JAR) after each merge.
- Enable safe and frequent releases of incremental features across the team.

## 6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core reminder-and-completion workflow.

### 6.1 Primary evaluation metric

**On-time completion rate:** the percentage of tracked obligations marked complete before their due date.

- Target: $\geq 85\%$ on-time completion among pilot users' tracked obligations.
- Attribution: measured from database timestamps (due date vs. completion timestamp).

### 6.2 Secondary evaluation metrics

- **Reminder delivery reliability:** percentage of scheduled reminders actually delivered on time by the scheduler.
- **Missed-deadline reduction:** self-reported reduction in missed deadlines compared to the user's prior (pre-app) baseline, collected via a short survey.
- **Task-completion latency:** time between the first reminder for an obligation and the user marking it complete.
- **OCR suggestion accuracy:** percentage of OCR-suggested dates (for supported document types) that the user accepts without correction.
- **Reliability:** the application launches and completes core flows (add/view/complete an obligation) without crashing across normal pilot usage.

### 6.3 Pilot validation plan *... high-level*

- Recruit 10–20 pilot users (peers/hostel residents) with genuine recurring obligations (subscriptions, ID renewals, fee deadlines, form submissions).
- Compare self-reported missed-deadline frequency before and after a short trial window, alongside in-system on-time completion data.

## 7 Scalability *... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of tracked obligations, uploaded documents, and supported document templates by:
   - keeping the UI, domain logic, and persistence layers decoupled (JavaFX / plain Java / JDBC),
   - indexing due-date queries in the database (the most frequent query pattern),
   - designing the OCR template library as a pluggable set of definitions, so adding a new document type does not require touching existing ones.
2. **Operationally deployable (practically):** The system should remain easy to run and distribute:
   - a single packaged, runnable JAR bundling the app and its dependencies,
   - a local SQLite database file, requiring no server setup,
   - CI pipeline for automated build and test on every change.

## 8 Engine availability heuristic

A mature set of "engines" (tooling/libraries) is available to implement this as a Java desktop project, within our team's current and developing skill set (Java, JDBC, JavaFX):

- JavaFX for the desktop UI (forms, tables, dialogs).
- JDBC with SQLite for local persistence of obligations, tasks, and reminder logs – no server infrastructure required.
- Tess4J (a Java wrapper for the open-source Tesseract OCR engine) for reading text out of a cropped document region.
- A lightweight in-process scheduler (`ScheduledExecutorService`) for periodic reminder checks – no external scheduling infrastructure required.
- JUnit for unit/integration tests on scheduling, recurrence, and OCR-region logic.
- A CI runner (e.g., GitHub Actions) for automated build and test on every push.

We will use these mature, well-documented components to focus on workflow design, scheduling correctness, and measurement rather than inventing new infrastructure, and to stay within our team's existing and developing skill set.

# 9 Project scope and deliverables *... iteration-friendly*

## 9.1 Initial deliverable *... first iteration*

- `Chore`/obligation data model and JDBC persistence layer (SQLite)
- Obligation entry form (name, category, due date, optional recurrence) in JavaFX
- Dashboard listing upcoming/overdue obligations
- In-app reminder generation for approaching deadlines
- Basic logging and timestamps for on-time completion measurement
- CI: build + unit tests on the persistence and scheduling logic

## 9.2 Subsequent deliverables

- Sub-task/checklist support within an obligation
- Optional document upload with OCR-based date detection, starting with a single document template (driving licence), with manual entry as the fallback
- Additional document templates (e.g., insurance renewal, loan EMI schedules)
- Pre-filled metadata templates for common renewal categories (e.g., ID documents, subscriptions, fee payments)
- Calendar-style view and improved search/filtering, with indexed due-date queries for scale readiness
- Optional deep-links to official service portals (e.g., DigiLocker) for the underlying document/transaction

# 10 Risks and mitigations

- **Privacy of uploaded documents:** document upload is optional, but even a temporarily-held photo of an ID document is sensitive; mitigate by only retaining what the user explicitly opts to keep, storing any retained files locally rather than on a remote server, and always asking before deletion vs. retention.
- **OCR misreads:** an incorrectly read date is worse than no date if left unconfirmed; mitigate by always requiring explicit user confirmation before an obligation is created from OCR, and by falling back cleanly to manual entry on low-confidence reads.
- **Limited document-type coverage:** template-based OCR only works for document layouts we have explicitly defined; mitigate by treating manual entry as the primary path and OCR as an incremental convenience layered on top, not a dependency for core functionality.
- **User adoption/habit formation:** a tracker only helps if people actually log obligations; mitigate with a fast, low-friction entry form and sensible category templates to reduce setup effort.
- **Reminder reliability:** a missed or late reminder defeats the product's purpose; mitigate with automated tests on the scheduling logic and clear in-app indicators for overdue items.

## 11 Summary

This project proposes a Java desktop application that helps individuals track and act on personal administrative deadlines and renewals, with an optional document-upload feature that uses template-based OCR to suggest due dates from supported document types (starting with a driving licence), always subject to user confirmation, with manual entry as the reliable default. It meets the course criteria by emphasizing time-to-value through rapid iterations, using CI to support frequent, safe, tested changes, and using measurable evaluation criteria (especially on-time completion rate and OCR suggestion accuracy). It remains focused on engineering workflow, scheduling correctness, and scalability readiness rather than research-driven novelty, and is scoped to build on our team's existing skills in SQL while developing our skills in Java, JDBC, and JavaFX.